

Design Decisions

The Library Management System presented in this project is a modular application designed to handle core operations in a library environment, including borrower registration, book searching, as well as fine management. The system is developed in Python, making use of GUI components to balance user accessibility with development and testing flexibility.

Furthermore, our GUI component was built using Python's tkinter library, which would act as the visual entry point for end users, where a window would display "Library System," as well as a button to register borrowers. The button is connected to `ui.borrow_form`, which manages the borrower registration process, which allows the GUI to be simple while also productive in delegating the actual registration process to backend logic, in which the borrower would have to input their ID and have it validated.

Adding on, a menu has been implemented that provides access to the wide range of operations available throughout the database, including searching for books, checking books in and out, managing fines, as well as exiting. Each of these operations are organized within the `lib` package, which includes many different Python modules that focus on each specific task, such as search, borrower, fines, and loaning. Moreover, `lib.search` possesses the `search_books` function that retrieves the records through its title, author, or ISBN. The `lib.borrower` function allows for the registration of a borrower with their personal details with `create_borrower()`. The `lib.fines` module has many functions that allows you to update (`update_fines()`), display (`display_fines()`), and pay (`pay_fines()`) all library fines. As for the menu functionality, it was deemed most appropriate to add all of the fine functions under a singular "Fines" button to ensure ease of use for the user. From there, the user can refresh, display, or pay the fines in question. "Refreshing"

finer simply makes use of the `update_finer()` function and fetches the most recent finer in the system, allowing the user to act on the most relevant data.

Lastly, the loaning function is focused on checking in as well as checking out books through `checkin()` and `checkout()`. These decisions were made with the architecture of the system to make the structure simple, organized, and reusable.

As for the schema, which is given on the last page of this document, there were some considerations to be made. While the key dependencies were given, it was imperative to implement them in a manner in which all dependencies are valid and all of the functionalities are operational. To ensure this, multiple files were created for the different functionalities, while two files were created for (1) creating the tables and (2) inserting sample data into the tables. With the addition of a GUI, a file structure also had to be formalized. This was done by placing all of the files related to database manipulation in a folder called “db”, all of the files with the core functionality in “lib”, and the files related to the UI portion in “ui”. This file structure enables easy editing and debugging for the developer. The entire database is stored in a file called “library”, with the file type of “db”.

FIGURE: SCHEMA

